

COMPARISONS AND BOOLEAN LOGIC

Turning data into yes/no decisions

```
record_count = 1_250
minimum_required = 1_000

has_enough_records = record_count
>= minimum_required
print(has_enough_records)
```

**True or False is enough to
drive decisions.**

```
is_ready = (
    is_active
    and has_approval
    and error_count == 0
)
```

30 min instruction • 30 min guided lab • record readiness challenge

The big idea: questions become values

A comparison is an expression that produces True or False.



Boolean logic lets a program check facts before it acts.

Boolean values: True and False

Python has a dedicated data type for yes/no answers.

True

False

```
is_active = True  
has_errors = False  
  
print(type(is_active))
```

`<class 'bool'>`

Capitalization matters: True and False are not the same as true and false.

Comparison operators

These operators ask questions about two values.

==

Equal to

`status == "active"`**!=**

Not equal to

`status != "closed"`**<**

Less than

`score < 70`**<=**

Less than or equal

`hours <= 1000`**>**

Greater than

`temp > 80`**>=**

Greater than or equal

`count >= minimum`

Equality versus assignment

One equal sign stores a value. Two equal signs compare values.

```
status = "active"
```

Assignment
store this value in this name

```
status == "active"
```

Comparison
ask whether these values match

Beginner trap: = changes data. == asks a question and returns True or False.

Numeric comparisons

Numbers can be compared with thresholds, limits, ranges, and requirements.

```
record_count = 1_250  
minimum_required = 1_000  
  
has_enough_records = record_count >=  
minimum_required  
  
print(has_enough_records)
```

$$1,250 \geq 1,000$$

True

Use clear variable names so the comparison reads like plain English.

String comparisons are exact

Case, spaces, and spelling all matter unless you normalize first.

```
print("Active" == "active")
```

False

```
status = " Active "  
normalized_status = status.strip().lower()  
is_active = normalized_status == "active"
```

True

Normalize first, then compare.

Membership testing with in

The in operator checks whether text appears inside other text.

```
file_name = "equipment_status.csv"

contains_status = "status" in file_name
is_csv = ".csv" in file_name

print(contains_status)
print(is_csv)
```

True
True

For filenames, prefer `.endswith(".csv")` when you specifically mean “extension is CSV.”

Combining conditions with and

and requires every condition to be True.

```
age = 35  
has_badge = True  
  
can_enter = age >= 18 and has_badge
```

age >= 18
True



has_badge
True

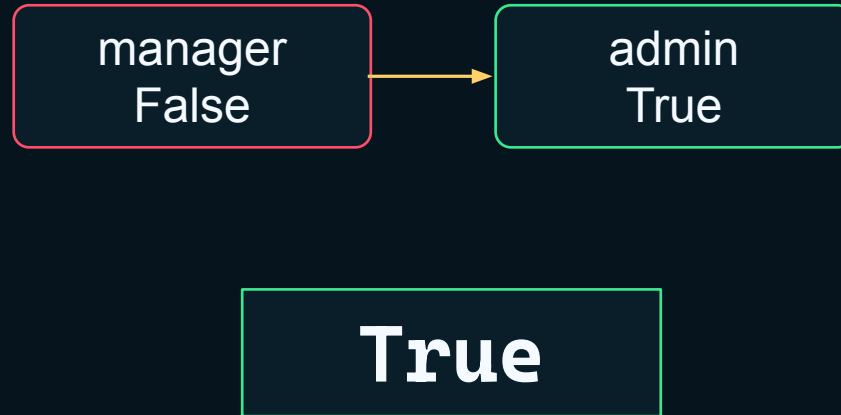
True

Useful pattern: all required checks must pass before the record is ready.

Combining conditions with or

or requires at least one condition to be True.

```
is_manager = False  
is_administrator = True  
  
has_elevated_access = is_manager or  
is_administrator
```



Useful pattern: one of several acceptable options is enough.

Negating a condition with not

not flips True to False or False to True.

```
system_locked = False  
system_available = not system_locked  
print(system_available)
```

not False → True

Use not carefully: double negatives can make code hard to read.

```
# Harder to read  
is_not_inactive = not is_active
```

Truth tables: the behavior of and / or

Truth tables make compound logic predictable.

A	B	A and B	A or B
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

and is only True on the bottom row. or is True when at least one input is True.

Operator precedence and parentheses

not runs before and; and runs before or. Parentheses make intent obvious.

```
can_process = is_active and has_approval or is_admin
```

Could be misunderstood

```
can_process = (is_active and has_approval) or is_admin
```

Clear intent

Beginner rule: use parentheses when combining more than two conditions.

Chained comparisons make ranges readable

Python can express “between” checks in a natural way.

```
temperature = 72  
  
is_normal = 60 <= temperature <= 80
```

$$60 \leq 72 \leq 80$$

```
# Equivalent, but longer  
is_normal = temperature >= 60 and temperature <= 80
```

Use chained comparisons for ranges: scores, temperatures, percentages, dates, and limits.

Clear Boolean variable names

Boolean names should sound like yes/no questions.

```
is_active = True  
has_required_fields = False  
can_process_record = True  
is_valid_temperature = False
```

is_

has_

can_

should_

Good prefixes make later
if statements read like
English.

Avoid vague names like flag, check, result, or ok when a clearer name is possible.

Demo: equipment readiness evaluator

Build several small Boolean checks, then combine them into one final result.

```
equipment_id = "EQ-1045"  
status = "active"  
operating_hours = 820  
maximum_hours = 1_000  
inspection_complete = True  
temperature = 74.5
```

```
has_equipment_prefix = equipment_id.startswith("EQ-")  
is_active = status == "active"  
is_within_hour_limit = operating_hours <=  
maximum_hours  
has_completed_inspection = inspection_complete  
is_temperature_normal = 60 <= temperature <= 80
```

Do not start with one giant expression. Name each smaller check first.

Combine checks into a final Boolean

The final value is True only when every required check passes.

```
is_ready = (  
    has_equipment_prefix  
    and is_active  
    and is_within_hour_limit  
    and has_completed_inspection  
    and is_temperature_normal  
)  
  
print(f"Ready for use: {is_ready}")
```

Correct identifier: True
Active status: True
Within hour limit: True
Inspection complete: True
Temperature normal: True

Ready for use: True

Guided exercise: print every intermediate condition

Students build the readiness evaluator from variables, comparisons, and Boolean logic.

```
equipment_id = "EQ-1045"  
status = "active"  
operating_hours = 820  
maximum_hours = 1_000  
inspection_complete = True  
temperature = 74.5
```

prefix begins with EQ-

status is active

hours under limit

inspection is complete

temperature is normal

final readiness

Lab rule: print the small checks before printing the big check.

Guided exercise: reporting the checks

Use f-strings so every condition has a readable label.

```
print(f"Correct identifier: {has_equipment_prefix}")
print(f"Active status: {is_active}")
print(f"Within hour limit: {is_within_hour_limit}")
print(f"Inspection complete:
{has_completed_inspection}")
print(f"Temperature normal: {is_temperature_normal}")
print(f"Ready for use: {is_ready}")
```

```
Correct identifier: True
Active status: True
Within hour limit: True
Inspection complete: True
Temperature normal: True
Ready for use: True
```

Readable output makes debugging logic much easier.

Additional practice questions

Write Boolean expressions that answer these questions.

Is a score between 70 and 100?

Is a user both active and authorized?

Is a quantity below the reorder level?

Is a year divisible by four?

Is a file a CSV or JSON file?

Is a record missing a required prefix?

Does a file name contain the word final?

```
is_score_valid = 70 <= score <= 100  
is_supported_file = file_name.endswith(".csv") or file_name.endswith(".json")
```

Applied challenge: record readiness evaluator

Create a final True/False result from several named checks.

identifier prefix

active status

$\geq 95\%$ complete

zero errors

approved

classification OK

supported type

Final value should be True only when every required condition passes.

Requirement: print every individual check and the final result.

Challenge starter code

Students fill in each Boolean expression and combine them.

```
record_id = "REC-2026-0042"  
status = "active"  
completion_percentage = 97.5  
error_count = 0  
approval_received = True  
security_classification = "public"  
file_extension = "csv"
```

```
has_correct_prefix = record_id.startswith("REC-")  
is_active = status == "active"  
is_complete_enough = completion_percentage >= 95  
has_no_errors = error_count == 0  
is_approved = approval_received  
is_classification_ok = security_classification ==  
"public" or security_classification == "internal"  
is_supported_type = file_extension == "csv" or  
file_extension == "json"
```

```
is_ready = (  
    has_correct_prefix and is_active and is_complete_enough  
    and has_no_errors and is_approved  
    and is_classification_ok and is_supported_type  
)
```

Challenge sample output and review

Answer reveal plus plain-language knowledge check.

```
Correct prefix: True
Active: True
Completion high enough: True
No errors: True
Approved: True
Classification acceptable: True
Supported type: True
Ready to process: True
```

What value does a comparison produce?

How is = different from ==?

When should you use and?

When should you use or?

Why are clear Boolean names useful?

Next step: use these True/False values to control program flow with if statements.